

공식 문서 기반 Skill / MCP 리뷰

대상: reverse-spec/SKILL.md , reverse-prd/SKILL.md 기준 문서(공식):
code.claude.com/docs/en/skills.md , .../commands.md , .../mcp.md 작성일:
2026-06-23

/ 검토 범위: SKILL.md 프론트매터 스펙 준수 여부, 디렉토리 구조, allowed-tools 포맷, MCP 도구 사용/표기 규칙. 공식 프론트매터 필드 표를 기준으로 사용했다.

1. 요약 (Verdict)

항목	판정
프론트매터 필드 유효성	통과 — 사용된 필드(name , description , argument-hint , allowed-tools) 모두 공식 스펙에 존재
잘못된/비공식 필드	없음 — allowedTools (camelCase), license , metadata 등 비공식 필드 미사용
디렉토리 구조	통과 — skill-name/SKILL.md 규칙 준수
allowed-tools 포맷	통과 — 콤마 구분 문자열 + Bash(pattern) 는 공식 지원 포맷
\$ARGUMENTS 사용	통과 — 공식 치환 변수
MCP 도구 표기	해당 없음 — MCP 도구를 쓰지 않으며, 현 기능상 불필요 (아래 5장 참조)

결론: 두 스킴은 공식 SKILL.md 스펙을 준수한다. 차단성 결함(blocking)은 없고, 아래는 개선 권고(권장/정보성) 항목이다.

2. 필드별 검증 (공식 프론트매터 표 대조)

공식 문서: "All fields are optional. Only `description` is recommended so Claude knows when to use the skill."

필드	현재 값	공식 스펙	판정
<code>name</code>	<code>reverse-spec</code> / <code>reverse-prd</code>	선택. 디렉토리 기반 스킬에서는 커맨드명이 디렉토리명에서 결정되고, <code>name</code> 은 <code>plugin-root SKILL.md</code> 에서만 권위를 가짐	동작상 무시됨(장식용). 디렉토리명과 일치하므로 무해
<code>description</code>	트리거 문구 포함 한국어 설명	권장 필드. 평문, listing에서 1,536자에서 잘림. 핵심 use case를 앞에	양호 (분량 충분히 짧음, 트리거 포함)
<code>argument-hint</code>	<code><파일_또는_디렉토리_경로> [--format ...]</code>	선택. 자동완성 시 표시되는 평문 힌트	유효
<code>allowed-tools</code>	<code>Read, Glob, Bash(find *), Bash(ls *), Bash(cat *), Bash(pip install *), Bash(python *)</code>	선택. 공백/콤마 구분 문자열 또는 YAML 리스트. <code>Bash(git add *)</code> 형태 패턴 허용	포맷 유효 (단, 보안 권고 → 4장)

YAML folded scalar(`>`)로 작성된 `description` / `allowed-tools` 는 단일 문자열로 접히며, 콤마 구분 문자열은 공식이 허용하는 포맷이라 문제없다.

3. 디렉토리 / 활성화 위치

- 현재: `reverse-spec/SKILL.md` , `reverse-prd/SKILL.md` (저장소 루트)
- 공식 프로젝트 스킬 경로: `.claude/skills/<dir>/SKILL.md` , 개인 스킬 경로: `~/.claude/skills/<dir>/SKILL.md`

권고 (R-1) — 적용됨: 이 저장소는 스킬을 배포/수집하는 컬렉션 성격이라 루트 배치는 합리적이다. 다만 이 스킬을 실제로 활성화하려면 `.claude/skills/` 아래에 있어야 자동 로드된다.

README에 설치 경로(예: `cp -r reverse-prd ~/.claude/skills/`)를 명시하거나, 프로젝트 내 활성화가 목적이면 `.claude/skills/` 로 이동할 것.

4. `allowed-tools` 보안 관점

공식 정의: "Tools Claude can use without per-use approval while skill is active." 즉 `allowed-tools` 에 올린 도구는 스킬 활성화 동안 개별 승인 프롬프트 없이 실행된다.

- `Bash(pip install *)` , `Bash(python *)` 는 사실상 임의 코드 실행을 사전 승인하는 셈이다.
- PDF/docx 생성을 위해 기능상 필요하긴 하나, 광범위한 권한이다.

권고 (**R-2**) — 적용됨: - 의도된 동작임을 SKILL.md 또는 README에 명시 (사용자가 권한 범위를 인지하도록). - 가능하면 범위를 좁힌다. 예: 생성 스크립트를 `scripts/` 로 분리하고

`Bash(python */scripts/render.py *)` 형태로 한정 (아래 R-3 참조). 주의: \$

{`CLAUDE_SKILL_DIR`} 치환은 스킬 본문에서만 동작하며 프론트매터 `allowed-tools` 에서는 확장되지 않는다(리터럴 문자열로 취급되어 어떤 명령과도 매칭 안 됨). 따라서 허가 패턴에는 경로 와일드카드를 사용해야 한다. - `pip install` 은 사전 설치를 전제하면 제거 가능 (런타임 설치 회피).

5. MCP 리뷰

현황: 두 스킬 모두 **MCP** 도구를 참조하지 않는다. 그리고 현 기능 범위에서는 그게 맞다. 역기획 문서 생성은 로컬 파일 읽기 + 로컬 PDF/docx 생성으로 완결되므로 외부 MCP 서버가 불필요하다.

향후 산출물을 외부 시스템에 게시(예: Confluence/Notion/Jira/Slack)하도록 확장한다면, 공식 MCP 도구 표기 규칙을 따라야 한다:

- 표준 MCP 서버: `mcp__<server-name>__<tool-name>` (예: `mcp_github_create_pr`)
- 플러그인 번들 MCP 서버: `mcp_plugin_<plugin-name>_<server-name>__<tool-name>`
(A-Z a-z 0-9 _ - 외 문자는 _ 로 치환)

이때 해당 도구를 `allowed-tools` 에 풀네임으로 추가한다. 예:

```
allowed-tools: >
  Read, Glob, Bash(python *),
  mcp__notion__create_page, mcp__confluence__create_content
```

권고 (**R-4**): 지금은 MCP 미사용이 적절. 외부 게시 기능을 넣을 때만 위 표기를 도입.

6. 구조 개선 권고 (선택)

권고 (R-3) — 적용됨: (이전) PDF/docx 생성 Python 코드가 SKILL.md 본문에 인라인으로 있었다. 공식 문서는 실행 스크립트를 `scripts/` 에 두고 `${CLAUDE_SKILL_DIR}` 로 참조하는 "supporting files" 구조를 권장한다. 분리 시 이점:

- SKILL.md가 가벼워져 모델이 지침을 더 잘 따른다.
- `allowed-tools` 를 `Bash(python */scripts/render.py *)` 로 좁혀 R-2 보안 권고도 충족. (프론트매터는 `${CLAUDE_SKILL_DIR}` 를 확장하지 않으므로 경로 와일드카드 패턴 사용.)

예시 구조:

```
reverse-prd/
├─ SKILL.md
├─ scripts/
│   ├─ render_pdf.py
│   └─ render_docx.py
└─ reference.md # Step 1 공통 파싱 규칙 (reverse-spec과 공유)
```

권고 (R-5) — 적용됨: (이전) `reverse-spec` 과 `reverse-prd` 의 Step 1(코드 파싱) 로직이 중복이었다. 공통 `reference.md` 로 분리하고 양쪽 SKILL.md에서 참조하면 유지보수성이 오른다.

7. 액션 아이템 정리

ID	구분	내용	우선 순위
R-1	적용됨	README에 설치 경로(<code>~/.claude/skills/</code> · <code>.claude/skills/</code>)·사용법·구조 명시	중
R-2	적용됨	<code>allowed-tools</code> 를 <code>Bash(python */scripts/render.py *)</code> 로 축소, <code>pip install</code> 도 패키지 한정	중
R-3	적용됨	인라인 Python을 <code>scripts/render.py</code> 로 분리, <code>\${CLAUDE_SKILL_DIR}</code> 참조로 호출	중
R-4	정보	MCP는 외부 게시 확장 시에만 <code>mcp_server_tool</code> 표기로 도입	하
R-5		Step 1 공통 파싱 로직을 <code>reference.md</code> 로 분리, 양쪽 SKILL.md에서 <code>\${CLAUDE_SKILL_DIR}/reference.md</code> 참조	하

ID	구분	내용	우선 순위
	적용됨		
—	정보	name 필드는 장식용(무해). 제거해도 됨	하

적용 이력 (2026-06-26): R-2·R-3·R-5는 리팩터링으로 반영됨 — 두 스킬이 `reference.md` (공통 파싱)와 `scripts/render.py` (공통 렌더러)를 공유하고, `allowed-tools` 는 스크립트 경로로 한정됨. R-1은 저장소 `README.md` 로 충족. R-4(MCP)는 현 기능상 불필요하여 정보성으로 유지.

정정 (2026-07-02): 최초 R-2 적용 시 사용한 `Bash(python ${CLAUDE_SKILL_DIR}/scripts/*)` 패턴은 프론트매터에서 변수가 확장되지 않아 어떤 명령과도 매칭되지 않는 사문이었다 (fail-closed — 보안 문제는 아니나 사전 승인 효과 없음). 공식 `permissions` 문서의 와일드카드 시맨틱(* 는 공백 포함 임의 문자열 매칭)에 따라 `Bash(python */scripts/render.py *)` 로 교체하여 실제로 매칭되는 최소 패턴으로 정정함.

출처

- <https://code.claude.com/docs/en/skills.md> (프론트매터 필드 표, 디렉토리 구조, `${CLAUDE_SKILL_DIR}`)
- <https://code.claude.com/docs/en/commands.md> (커맨드↔스킬 통합, `$ARGUMENTS`)
- <https://code.claude.com/docs/en/mcp.md> (MCP 도구 네이밍 `mcp__server__tool`)